Отчёт по 2 этапу курсовой работы

по дисциплине «Информационные системы»

Выполнили: Гладышев Иван
Хлопцов Сергей

Руководитель: Кобик Никита Алексеевич

Санкт-Петербург, 2026 г.

1. ER-модель базы данных

1.1. Выявленные сущности

На основе описания предметной области и прецедентов UC-1...UC-7 выделены следующие сущности:

1. **User (Пользователь)** – учётная запись сотрудника организации (администратора или менеджера).
2. **Role (Роль)** – справочник ролей (администратор организации, менеджер по продажам / юрист).
3. **Client (Клиент)** – физическое или юридическое лицо, обратившееся за юридической услугой.
4. **Agent (Агент)** – внешний посредник, приводящий клиентов в юридическую фирму.
5. **Lead (Лид)** – обращение клиента, зарегистрированное в системе.
6. **AgentOffer (Предложение агента)** – конкретное финансовое предложение агента по приведённому им лиду (комиссия, сумма сделки).
7. **Opponent (Оппонент)** – компания или физическое лицо, выступающее противоположной стороной по делу.
8. **Affiliation (Аффилиация)** – связь «материнская компания – дочерняя структура» между двумя организациями.
9. **ConflictCheck (Проверка конфликта интересов)** – результат автоматической проверки оппонента по базе клиентов и аффилиаций.
10. **BusinessRule (Бизнес-правило)** – декларативное правило в формате JSON, настраиваемое администратором через визуальный конструктор.

11. **LlmSetting (Настройка LLM)** – параметры подключения к внешнему LLM API (провайдер, токен, базовый URL).

12. **LicenseActivation (Активация лицензии)** – данные привязки лицензии к конкретной установке системы.

1.2. Отношения между сущностями

Все отношения между сущностями приведены ниже. Отношения вида «многие-ко-многим» реализуются через таблицы-связки, которые одновременно являются самостоятельными сущностями модели. Для обеспечения аудита действий и разграничения прав (NFR-2) все бизнес-сущности связаны с пользователем, создавшим или изменившим запись.

- **Agent ↔ Lead** – «многие-ко-многим». Один агент может привести несколько лидов, а один лид может быть приведён несколькими агентами одновременно. Реализуется через сущность-ассоциацию **AgentOffer**, хранящую финансовое предложение агента и срок действия его приоритета.

- **Client ↔ Opponent** – «многие-ко-многим». Один клиент может участвовать в делах с разными оппонентами, а один оппонент может встречаться в делах разных клиентов. Реализуется через сущность **Lead**.

- **AgentOffer → Lead** – «многие-к-одному». Каждое предложение относится ровно к одному лиду; один лид может иметь несколько предложений от разных агентов.

- **AgentOffer → Agent** – «многие-к-одному». Каждое предложение исходит от одного агента; один агент может иметь множество предложений по разным лидам.

- **Lead → Client** – «многие-к-одному». Каждый лид относится к одному клиенту; у одного клиента может быть несколько лидов.

- **Lead → Opponent** – «многие-к-одному». Каждый лид имеет одного оппонента; один оппонент может фигурировать в разных лидах.

- **ConflictCheck → Lead** – «многие-к-одному». Каждая проверка привязана к одному лиду; по одному лиду может быть выполнено несколько проверок (история).
- **ConflictCheck → Opponent** – «многие-к-одному». Каждая проверка выполняется по одному оппоненту; один оппонент может проверяться в разных лидах.
- **Affiliation → Opponent** (parent) – «многие-к-одному». Каждая запись об аффилиации указывает на одну материнскую организацию.
- **Affiliation → Opponent** (child) – «многие-к-одному». Каждая запись об аффилиации указывает на одну дочернюю организацию.
- **User → Role** – «многие-к-одному». Каждый пользователь имеет ровно одну роль; одна роль может быть назначена многим пользователям.
- **User → LicenseActivation** – «один-ко-многим». Один администратор может выполнить активацию нескольких установок системы (например, при переустановке или развёртывании на разных серверах). Реализуется внешним ключом `license_activations.activated_by_user_id`.
- **Client → Client** (self-reference) – «один-ко-многим». Поле `merged_into_id` позволяет связывать дубликаты клиентов с основной записью после объединения.
- **User → Lead** / **Client** / **Agent** / **AgentOffer** / **ConflictCheck** / **BusinessRule** / **LlmSetting** – «один-ко-многим». Для обеспечения аудита и безопасности (NFR-2) все ключевые бизнес-сущности содержат поля `created_by` и/или `updated_by`, ссылающиеся на пользователя, совершившего действие. Это позволяет отслеживать авторство записей и разграничивать доступ (например, менеджер видит только свои лиды).

1.3. Диаграмма ER-модели

На рисунке 1.1 представлена ER-диаграмма, построенная в нотации Чена. Прямоугольники обозначают сущности, ромбы – связи, овалы – ключевые атрибуты. Числа у ромбов указывают мощность связи (1, M, N).

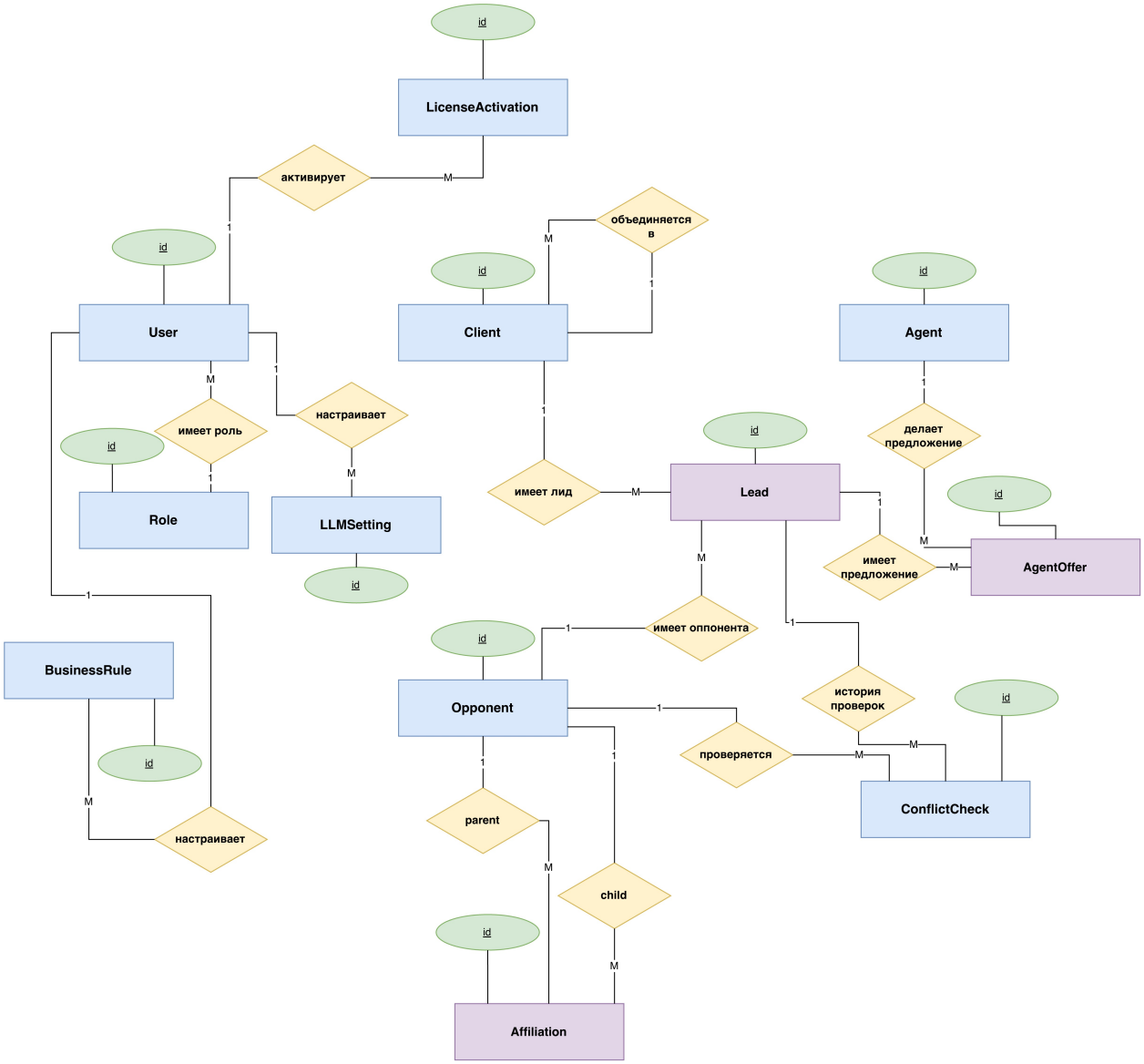


Рис. 1.1: ER-модель базы данных

2. Даталогическая модель

ER-модель преобразована в даталогическую модель согласно стандартным правилам: каждая сущность становится таблицей, каждое отношение «многие-ко-многим» – отдельной таблицей-связке, отношения «один-ко-многим» реализуются через внешние ключи.

2.1. Описание таблиц

В таблице 2.1 приведён перечень таблиц с указанием первичного ключа, основных атрибутов и внешних ключей.

Таблица	**PK**	**Основные атрибуты**	**Внешние ключи**
roles	id	name, code, description	–
users	id	email, password_hash, full_name, role_id, is_active, created_at	role_id → roles(id)
clients	id	client_type, full_name, short_name, inn, ogrn, phone, email, address, birth_date, merged_into_id, created_by, updated_by, created_at, updated_at	merged_into_id → clients(id), created_by → users(id), updated_by → users(id)
agents	id	full_name, phone, email, commission_default, is_active, created_by, created_at	created_by → users(id)

leads	id	client_id, opponent_id, description, amount, status, created_by, updated_by, created_at, updated_at	client_id → clients(id), opponent_id → opponents(id), created_by → users(id), updated_by → users(id)
agent_offers	id	lead_id, agent_id, amount, commission, priority_expires_at, is_selected, created_by, created_at	lead_id → leads(id), agent_id → agents(id), created_by → users(id)
opponents	id	name, normalized_name, inn, ogrn, is_individual, created_by, created_at	created_by → users(id)
affiliations	id	parent_id, child_id, relation_type, created_by, created_at	parent_id → opponents(id), child_id → opponents(id), created_by → users(id)
conflict_checks	id	lead_id, opponent_id, result, reason, checked_by, checked_at	lead_id → leads(id), opponent_id → opponents(id), checked_by → users(id)
business_rules	id	rule_type, name, definition (JSONB), is_active, created_by, updated_by, created_at, updated_at	created_by → users(id), updated_by → users(id)
llm_settings	id	provider, base_url, model, api_token_encrypted, is_active, updated_by, updated_at	updated_by → users(id)

license_activations	id	installation_id, license_key_hash, activation_secret, token, token_expires_at, activated_by_user_id, activated_at	activated_by_user_id → users(id)

2.2. Схема базы данных

На рисунке 2.1 представлена даталогическая схема базы данных с указанием типов данных, первичных ключей (PK) и внешних ключей (FK).

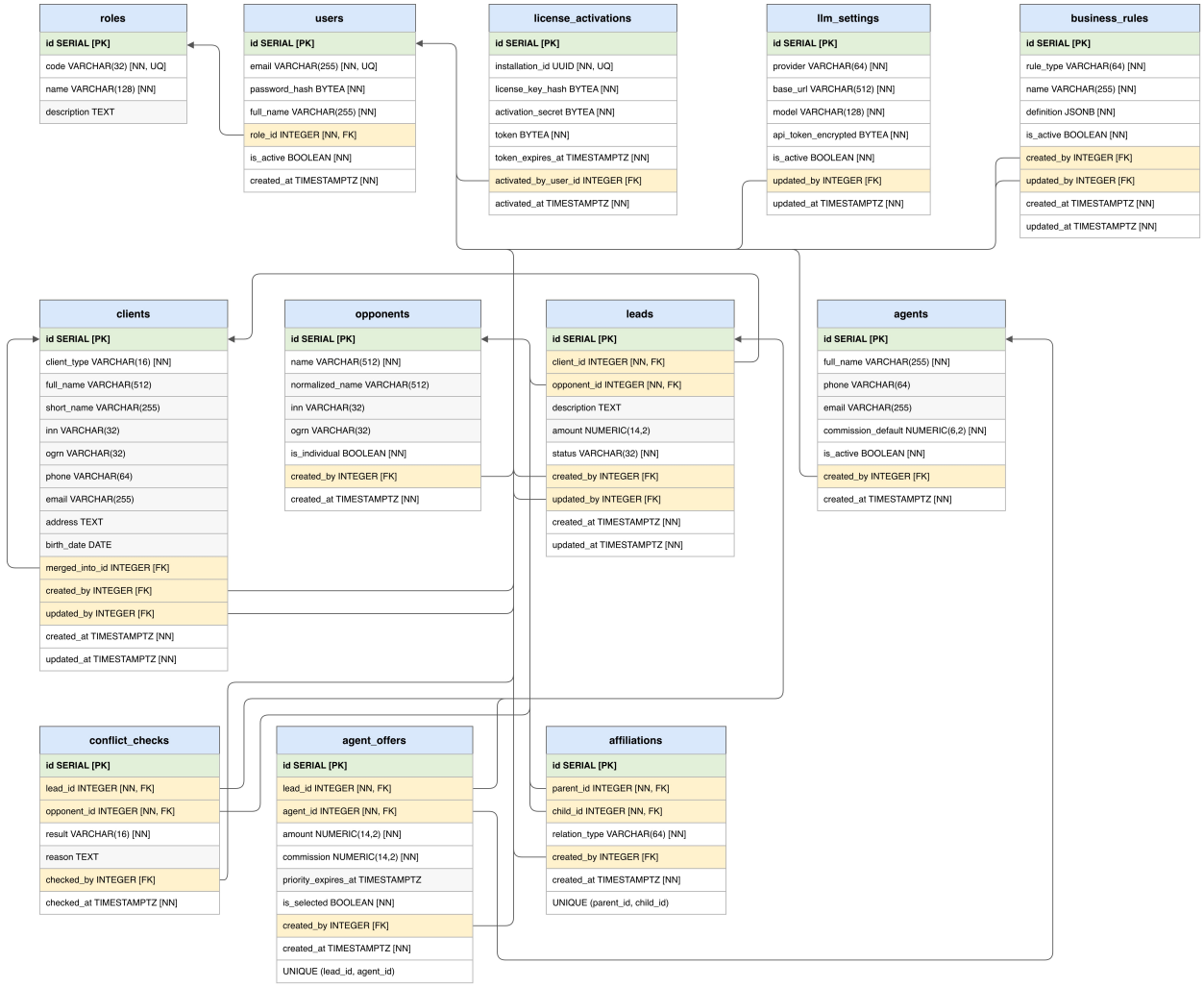


Рис. 2.1: Даталогическая схема базы данных

3. Реализация схемы в PostgreSQL

3.1. Скрипт создания базы данных

```
DROP DATABASE IF EXISTS lawfirm_db;
CREATE DATABASE lawfirm_db;
\c lawfirm_db

CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE TABLE roles (
  id SERIAL PRIMARY KEY,
  code VARCHAR(32) NOT NULL UNIQUE,
  name VARCHAR(128) NOT NULL,
  description TEXT
);

CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) NOT NULL UNIQUE,
  password_hash BYTEA NOT NULL,
  full_name VARCHAR(255) NOT NULL,
  role_id INTEGER NOT NULL REFERENCES roles(id) ON DELETE
     RESTRICT,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_users_email ON users (email);
CREATE INDEX idx_users_role ON users (role_id);

CREATE TABLE clients (
  id SERIAL PRIMARY KEY,
  client_type VARCHAR(16) NOT NULL CHECK (client_type IN ('
     individual', 'legal')),
  full_name VARCHAR(512),
  short_name VARCHAR(255),
  inn VARCHAR(32),
  ogrn VARCHAR(32),
```

```
  phone VARCHAR(64),
  email VARCHAR(255),
  address TEXT,
  birth_date DATE,
  merged_into_id INTEGER REFERENCES clients(id) ON DELETE SET NULL,
  created_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
  updated_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_clients_phone ON clients (phone) WHERE phone IS NOT NULL;
CREATE INDEX idx_clients_email ON clients (email) WHERE email IS NOT NULL;
CREATE INDEX idx_clients_inn ON clients (inn) WHERE inn IS NOT NULL;
CREATE INDEX idx_clients_full_name ON clients (full_name);
CREATE INDEX idx_clients_merged ON clients (merged_into_id) WHERE merged_into_id IS NOT NULL;
CREATE INDEX idx_clients_created_by ON clients (created_by);
CREATE INDEX idx_clients_updated_by ON clients (updated_by);

CREATE TABLE agents (
  id SERIAL PRIMARY KEY,
  full_name VARCHAR(255) NOT NULL,
  phone VARCHAR(64),
  email VARCHAR(255),
  commission_default NUMERIC(6,2) NOT NULL DEFAULT 0,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_agents_phone ON agents (phone) WHERE phone IS NOT NULL;
CREATE INDEX idx_agents_email ON agents (email) WHERE email IS NOT NULL;
CREATE INDEX idx_agents_created ON agents (created_by);

CREATE TABLE opponents (
  id SERIAL PRIMARY KEY,
```

```
  name VARCHAR(512) NOT NULL,
  normalized_name VARCHAR(512),
  inn VARCHAR(32),
  ogrn VARCHAR(32),
  is_individual BOOLEAN NOT NULL DEFAULT FALSE,
  created_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_opponents_inn ON opponents (inn) WHERE inn IS NOT NULL;
CREATE INDEX idx_opponents_norm ON opponents (normalized_name);
CREATE INDEX idx_opponents_created ON opponents (created_by);

CREATE TABLE affiliations (
  id SERIAL PRIMARY KEY,
  parent_id INTEGER NOT NULL REFERENCES opponents(id) ON DELETE CASCADE,
  child_id INTEGER NOT NULL REFERENCES opponents(id) ON DELETE CASCADE,
  relation_type VARCHAR(64) NOT NULL DEFAULT 'subsidiary',
  created_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_affiliation UNIQUE (parent_id, child_id),
  CONSTRAINT chk_no_self_affil CHECK (parent_id <> child_id)
);
CREATE INDEX idx_affiliations_parent ON affiliations (parent_id);
CREATE INDEX idx_affiliations_child ON affiliations (child_id);
CREATE INDEX idx_affiliations_created ON affiliations (created_by);

CREATE TABLE leads (
  id SERIAL PRIMARY KEY,
  client_id INTEGER NOT NULL REFERENCES clients(id) ON DELETE RESTRICT,
  opponent_id INTEGER NOT NULL REFERENCES opponents(id) ON DELETE RESTRICT,
  description TEXT,
  amount NUMERIC(14,2),
  status VARCHAR(32) NOT NULL DEFAULT 'new' CHECK (status IN ('new', 'checking', 'active', 'closed', 'cancelled')),
```

```
  created_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
  updated_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_leads_client ON leads (client_id);
CREATE INDEX idx_leads_opponent ON leads (opponent_id);
CREATE INDEX idx_leads_status ON leads (status);
CREATE INDEX idx_leads_created ON leads (created_at DESC);
CREATE INDEX idx_leads_created_by ON leads (created_by);
CREATE INDEX idx_leads_updated_by ON leads (updated_by);

CREATE TABLE agent_offers (
  id SERIAL PRIMARY KEY,
  lead_id INTEGER NOT NULL REFERENCES leads(id) ON DELETE
    CASCADE,
  agent_id INTEGER NOT NULL REFERENCES agents(id) ON DELETE
    RESTRICT,
  amount NUMERIC(14,2) NOT NULL,
  commission NUMERIC(14,2) NOT NULL DEFAULT 0,
  priority_expires_at TIMESTAMPTZ,
  is_selected BOOLEAN NOT NULL DEFAULT FALSE,
  created_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_offer_per_lead_agent UNIQUE (lead_id, agent_id)
);
CREATE INDEX idx_offers_lead ON agent_offers (lead_id);
CREATE INDEX idx_offers_agent ON agent_offers (agent_id);
CREATE INDEX idx_offers_expires ON agent_offers (
   priority_expires_at) WHERE priority_expires_at IS NOT NULL;
CREATE INDEX idx_offers_selected ON agent_offers (lead_id)
   WHERE is_selected = TRUE;
CREATE INDEX idx_offers_created ON agent_offers (created_by);

CREATE TABLE conflict_checks (
  id SERIAL PRIMARY KEY,
  lead_id INTEGER NOT NULL REFERENCES leads(id) ON DELETE
    CASCADE,
  opponent_id INTEGER NOT NULL REFERENCES opponents(id) ON
    DELETE RESTRICT,
```

```
135   result VARCHAR(16) NOT NULL CHECK (result IN ('ok', 'warning'
          , 'blocked')),
136   reason TEXT,
137   checked_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
138   checked_at TIMESTAMPTZ NOT NULL DEFAULT now()
139 );
140 CREATE INDEX idx_conflicts_lead ON conflict_checks (lead_id);
141 CREATE INDEX idx_conflicts_opponent ON conflict_checks (
       opponent_id);
142 CREATE INDEX idx_conflicts_at ON conflict_checks (checked_at
       DESC);
143 CREATE INDEX idx_conflicts_checked ON conflict_checks (
       checked_by);
144
145 CREATE TABLE business_rules (
146   id SERIAL PRIMARY KEY,
147   rule_type VARCHAR(64) NOT NULL,
148   name VARCHAR(255) NOT NULL,
149   definition JSONB NOT NULL,
150   is_active BOOLEAN NOT NULL DEFAULT TRUE,
151   created_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
152   updated_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
153   created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
154   updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
155 );
156 CREATE INDEX idx_rules_type ON business_rules (rule_type);
157 CREATE INDEX idx_rules_active ON business_rules (is_active)
       WHERE is_active = TRUE;
158 CREATE INDEX idx_rules_def ON business_rules USING GIN (
       definition);
159 CREATE INDEX idx_rules_created ON business_rules (created_by);
160 CREATE INDEX idx_rules_updated ON business_rules (updated_by);
161
162 CREATE TABLE llm_settings (
163   id SERIAL PRIMARY KEY,
164   provider VARCHAR(64) NOT NULL,
165   base_url VARCHAR(512) NOT NULL,
166   model VARCHAR(128) NOT NULL,
167   api_token_encrypted BYTEA NOT NULL,
168   is_active BOOLEAN NOT NULL DEFAULT FALSE,
169   updated_by INTEGER REFERENCES users(id) ON DELETE SET NULL,
```

```
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_llm_updated ON llm_settings (updated_by);

CREATE TABLE license_activations (
  id SERIAL PRIMARY KEY,
  installation_id UUID NOT NULL UNIQUE,
  license_key_hash BYTEA NOT NULL,
  activation_secret BYTEA NOT NULL,
  token BYTEA NOT NULL,
  token_expires_at TIMESTAMPTZ NOT NULL,
  activated_by_user_id INTEGER REFERENCES users(id) ON DELETE
     SET NULL,
  activated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_license_expires ON license_activations (
   token_expires_at);
CREATE INDEX idx_license_user ON license_activations (
   activated_by_user_id);
```

Листинг 3.1: create_database.sql – создание БД и схемы

3.2. Скрипт удаления базы данных

```
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'lawfirm_db'
  AND pid <> pg_backend_pid();

DROP DATABASE IF EXISTS lawfirm_db;
```

Листинг 3.2: drop_database.sql – полное удаление БД

4. Обеспечение целостности данных

Помимо стандартных ограничений (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK), целостность бизнес-данных обеспечивается триггерами.

4.1. Триггер автоматического обновления updated_at

```
CREATE OR REPLACE FUNCTION fn_set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_leads_updated
    BEFORE UPDATE ON leads
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();

CREATE TRIGGER trg_clients_updated
    BEFORE UPDATE ON clients
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();

CREATE TRIGGER trg_rules_updated
    BEFORE UPDATE ON business_rules
    FOR EACH ROW EXECUTE FUNCTION fn_set_updated_at();
```

Листинг 4.1: Триггер set_updated_at

4.2. Триггер расчёта срока приоритета агента

При создании предложения агента автоматически вычисляется срок действия приоритета на основе активного бизнес-правила типа `agent_priority_duration_days` (по умолчанию 90 дней).

```
CREATE OR REPLACE FUNCTION fn_set_priority_expires()
RETURNS TRIGGER AS $$
DECLARE
    v_days INT;
```

```
BEGIN
    SELECT COALESCE(
        (definition->>'days')::INT,
        90
    )
    INTO v_days
    FROM business_rules
    WHERE rule_type = 'agent_priority_duration_days'
      AND is_active = TRUE
    LIMIT 1;

    NEW.priority_expires_at := now() + (v_days  ' days')::
       INTERVAL;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_set_priority_expires
    BEFORE INSERT ON agent_offers
    FOR EACH ROW
    WHEN (NEW.priority_expires_at IS NULL)
    EXECUTE FUNCTION fn_set_priority_expires();
```

Листинг 4.2: Триггер trg_set_priority_expires

4.3. Триггер защиты от удаления клиентов с активными лидами

```
CREATE OR REPLACE FUNCTION fn_prevent_client_delete()
RETURNS TRIGGER AS $$
DECLARE
    v_cnt INT;
BEGIN
    SELECT COUNT(*) INTO v_cnt
    FROM leads
    WHERE client_id = OLD.id
      AND status IN ('new', 'checking', 'active');

    IF v_cnt > 0 THEN
        RAISE EXCEPTION
            'Нельзя удалить клиента: есть % активных лидов',
```

```
            v_cnt
            USING ERRCODE = '23503';
    END IF;
    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_client_delete
    BEFORE DELETE ON clients
    FOR EACH ROW EXECUTE FUNCTION fn_prevent_client_delete();
```

Листинг 4.3: Триггер защиты клиентов

5. Заполнение тестовыми данными

Скрипт `test_data.sql` заполняет базу данными, покрывающими все основные прецеденты UC-1...UC-7.

```sql
BEGIN;

-- ============================================================
-- Очистка тестовых данных
-- ============================================================

TRUNCATE TABLE
    conflict_checks,
    agent_offers,
    leads,
    affiliations,
    license_activations,
    llm_settings,
    business_rules,
    opponents,
    agents,
    clients,
    users,
    roles
RESTART IDENTITY CASCADE;


-- ============================================================
-- Роли
-- ============================================================

INSERT INTO roles (code, name, description) VALUES
(
    'ADMIN',
    'Администратор',
    'Управление пользователями, правилами, LLM и лицензией'
),
(
    'MANAGER',
    'Менеджер по продажам',
```

```
    'Работа с клиентами, лидами, агентами и конфликтами'
);


-- ============================================================
-- Пользователи
--
-- BYTEA ниже -- тестовые заглушки.
-- Реальное хеширование пароля выполняется приложением.
-- ============================================================

INSERT INTO users (
    email,
    password_hash,
    full_name,
    role_id
)
VALUES
(
    'admin@lawfirm.test',
    decode('01020304', 'hex'),
    'Кобик Никита',
    (SELECT id FROM roles WHERE code = 'ADMIN')
),
(
    'manager1@lawfirm.test',
    decode('05060708', 'hex'),
    'Иван Гладышев',
    (SELECT id FROM roles WHERE code = 'MANAGER')
),
(
    'manager2@lawfirm.test',
    decode('090a0b0c', 'hex'),
    'Сергей Хлопцов',
    (SELECT id FROM roles WHERE code = 'MANAGER')
);


-- ============================================================
-- Бизнес-правила
-- ============================================================
```

```sql

INSERT INTO business_rules (
    rule_type,
    name,
    definition,
    is_active,
    created_by,
    updated_by
)
VALUES
(
    'agent_priority_duration_days',
    'Срок приоритета агента',
    '{"days": 90}'::jsonb,
    TRUE,
    (SELECT id FROM users WHERE email = 'admin@lawfirm.test'),
    (SELECT id FROM users WHERE email = 'admin@lawfirm.test')
),
(
    'conflict_of_interest',
    'Проверка текущих и недавних клиентов',
    '{"archive_years": 5}'::jsonb,
    TRUE,
    (SELECT id FROM users WHERE email = 'admin@lawfirm.test'),
    (SELECT id FROM users WHERE email = 'admin@lawfirm.test')
),
(
    'commission_max_percent',
    'Максимальная комиссия агента',
    '{"max_percent": 50}'::jsonb,
    TRUE,
    (SELECT id FROM users WHERE email = 'admin@lawfirm.test'),
    (SELECT id FROM users WHERE email = 'admin@lawfirm.test')
);


-- ============================================================
-- Агенты
-- ============================================================

INSERT INTO agents (
```

```
118     full_name,
119     phone,
120     email,
121     commission_default,
122     created_by
123 )
124 VALUES
125 (
126     'Агентик мой любимый.',
127     '+79001168333',
128     'lalala@agent.test',
129     10.00,
130     (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
        )
131 ),
132 (
133     'ООО "ЮрПосредник"',
134     '+79004445566',
135     'info@jurposred.test',
136     15.00,
137     (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
        )
138 ),
139 (
140     'ИП "Иванова М.И."',
141     '+79007778899',
142     'ivanova@agent.test',
143     12.00,
144     (SELECT id FROM users WHERE email = 'manager2@lawfirm.test'
        )
145 );
146
147
148 -- ================================================================
149 -- Клиенты
150 -- ================================================================
151
152 -- Обычный физический клиент.
153 INSERT INTO clients (
154     client_type,
155     full_name,
```

```
156     phone,
157     email,
158     created_by,
159     updated_by
160 )
161 VALUES
162 (
163     'individual',
164     'Смирнов Андрей',
165     '+79111234567',
166     'zirota@test.ru',
167     (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
        ),
168     (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
        )
169 );
170
171 -- Намеренный дубликат Смирнова:
172 -- совпадает телефон, имя и почта записаны иначе.
173 INSERT INTO clients (
174     client_type,
175     full_name,
176     phone,
177     email,
178     created_by,
179     updated_by
180 )
181 VALUES
182 (
183     'individual',
184     'Смирнов А. П.',
185     '+79111234567',
186     'a.smirnov@test.ru',
187     (SELECT id FROM users WHERE email = 'manager2@lawfirm.test'
        ),
188     (SELECT id FROM users WHERE email = 'manager2@lawfirm.test'
        )
189 );
190
191 -- Юридическое лицо.
192 INSERT INTO clients (
```

```
    client_type,
    short_name,
    inn,
    ogrn,
    phone,
    email,
    created_by,
    updated_by
)
VALUES
(

    'legal',
    'ООО "Бнал".',
    '7701234567',
    '1027700000001',
    '+79001112233',
    'info@stroyinvest.test',
    (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
       ),
    (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
       )
);

-- Второй намеренный дубликат:
-- тот же ИНН, другое написание.
INSERT INTO clients (
    client_type,
    short_name,
    inn,
    created_by,
    updated_by
)
VALUES
(
    'legal',
    'компания обнал',
    '7701234567',
    (SELECT id FROM users WHERE email = 'manager2@lawfirm.test'
       ),
    (SELECT id FROM users WHERE email = 'manager2@lawfirm.test'
```

```
        )
);

-- Эта организация является действующим клиентом фирмы
-- и будет использоваться для проверки конфликта.
INSERT INTO clients (
    client_type,
    short_name,
    inn,
    ogrn,
    phone,
    email,
    created_by,
    updated_by
)
VALUES
(
    'legal',
    'ПАО Сбербанк',
    '7707083893',
    '1027700132195',
    '+74950000000',
    'client@sber.test',
    (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
        ),
    (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
        )
);

-- Клиент с неполными данными:
-- e-mail и телефон отсутствуют.
INSERT INTO clients (
    client_type,
    short_name,
    inn,
    created_by,
    updated_by
)
VALUES
(
    'legal',
```

```
269     'ООО "БезПочты"',
270     '7812345678',
271     (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
            ),
272     (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
            )
273 );
274
275
276 -- ================================================================
277 -- Оппоненты
278 -- ================================================================
279
280 INSERT INTO opponents (
281     name,
282     normalized_name,
283     inn,
284     ogrn,
285     is_individual,
286     created_by
287 )
288 VALUES
289 (
290     'ПАО Сбербанк',
291     'сбербанк',
292     '7707083893',
293     '1027700132195',
294     FALSE,
295     (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
            )
296 ),
297 (
298     'АО "СберЛогистика"',
299     'сберлогистика',
300     '7707000001',
301     '1027700000002',
302     FALSE,
303     (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
            )
304 ),
305 (
```

```
306     'ООО "Конкурент-1"',
307     'конкурент 1',
308     '7799887766',
309     '1027700000003',
310     FALSE,
311     (SELECT id FROM users WHERE email = 'manager1@lawfirm.test'
           )
312 ),
313 (
314     'Николаев Виктор Сергеевич',
315     'николаев виктор сергеевич',
316     NULL,
317     NULL,
318     TRUE,
319     (SELECT id FROM users WHERE email = 'manager2@lawfirm.test'
           )
320 );
321
322
323 -- ============================================================
324 -- Аффилированность организаций
325 --
326 -- СберЛогистика является дочерней организацией Сбербанка.
327 -- При этом СберЛогистика сама НЕ находится в clients.
328 -- Поэтому конфликт по ней будет именно конфликтом
329 -- через аффилированность, а не прямым совпадением.
330 -- ============================================================
331
332 INSERT INTO affiliations (
333     parent_id,
334     child_id,
335     relation_type,
336     created_by
337 )
338 VALUES
339 (
340     (SELECT id
341        FROM opponents
342       WHERE normalized_name = 'сбербанк'),
343
344     (SELECT id
```

```
345         FROM opponents
346        WHERE normalized_name = 'сберлогистика'),
347
348     'subsidiary',
349
350     (SELECT id
351        FROM users
352       WHERE email = 'admin@lawfirm.test')
353 );
354
355
356 -- ============================================================
357 -- Лиды
358 -- ============================================================
359
360 -- Существующий активный лид Сбербанка.
361 -- Он показывает, что Сбербанк является текущим клиентом фирмы.
362 INSERT INTO leads (
363     client_id,
364     opponent_id,
365     description,
366     amount,
367     status,
368     created_by,
369     updated_by,
370     created_at,
371     updated_at
372 )
373 VALUES
374 (
375     (SELECT id FROM clients
376       WHERE short_name = 'ПАО Сбербанк'),
377
378     (SELECT id FROM opponents
379       WHERE normalized_name = 'конкурент 1'),
380
381     'Текущее юридическое сопровождение Сбербанка',
382     1000000.00,
383     'active',
384
385     (SELECT id FROM users
```

```
386         WHERE email = 'manager1@lawfirm.test'),
387
388     (SELECT id FROM users
389         WHERE email = 'manager1@lawfirm.test'),
390
391     now() - INTERVAL '30 days',
392     now() - INTERVAL '1 day'
393 );
394
395
396 -- UC-2: прямой конфликт.
397 INSERT INTO leads (
398     client_id,
399     opponent_id,
400     description,
401     amount,
402     status,
403     created_by,
404     updated_by
405 )
406 VALUES
407 (
408     (SELECT id FROM clients
409         WHERE ogrn = '1027700000001'),
410
411     (SELECT id FROM opponents
412         WHERE normalized_name = 'сбербанк'),
413
414     'Спор против Сбербанка',
415     300000.00,
416     'checking',
417
418     (SELECT id FROM users
419         WHERE email = 'manager1@lawfirm.test'),
420
421     (SELECT id FROM users
422         WHERE email = 'manager1@lawfirm.test')
423 );
424
425
426 -- UC-2: конфликт через аффилированность.
```

```sql
INSERT INTO leads (
    client_id,
    opponent_id,
    description,
    amount,
    status,
    created_by,
    updated_by
)
VALUES
(
    (SELECT id FROM clients
      WHERE ogrn = '1027700000001'),

    (SELECT id FROM opponents
      WHERE normalized_name = 'сберлогистика'),

    'Спор против дочерней структуры клиента',
    1500000.00,
    'checking',

    (SELECT id FROM users
      WHERE email = 'manager1@lawfirm.test'),

    (SELECT id FROM users
      WHERE email = 'manager1@lawfirm.test')
);


-- UC-2: конфликта нет.
-- Этот же лид используется для UC-4.
INSERT INTO leads (
    client_id,
    opponent_id,
    description,
    amount,
    status,
    created_by,
    updated_by
)
VALUES
```

```
(
    (SELECT id FROM clients
      WHERE full_name = 'Смирнов Андрей'),

    (SELECT id FROM opponents
      WHERE normalized_name = 'конкурент 1'),

    'Защита товарного знака',
    500000.00,
    'new',

    (SELECT id FROM users
      WHERE email = 'manager2@lawfirm.test'),

    (SELECT id FROM users
      WHERE email = 'manager2@lawfirm.test')
);


-- ============================================================
-- Предложения агентов
-- ============================================================

-- Действующий приоритет.
-- priority_expires_at заполнит триггер
INSERT INTO agent_offers (
    lead_id,
    agent_id,
    amount,
    commission,
    is_selected,
    created_by
)
VALUES
(
    (SELECT id FROM leads
      WHERE description = 'Защита товарного знака'),

    (SELECT id FROM agents
      WHERE email = 'lalala@agent.test'),

```

```
    500000.00,
    40000.00,
    TRUE,

    (SELECT id FROM users
      WHERE email = 'manager2@lawfirm.test')
);


-- Второе действующее предложение.
INSERT INTO agent_offers (
    lead_id,
    agent_id,
    amount,
    commission,
    is_selected,
    created_by
)
VALUES
(
    (SELECT id FROM leads
      WHERE description = 'Защита товарного знака'),

    (SELECT id FROM agents
      WHERE email = 'info@jurposred.test'),

    520000.00,
    80000.00,
    FALSE,

    (SELECT id FROM users
      WHERE email = 'manager2@lawfirm.test')
);


-- Намеренно просроченный приоритет.
-- Чистая прибыль здесь самая высокая,
-- но приоритет агента уже недействителен.
INSERT INTO agent_offers (
    lead_id,
    agent_id,
```

```
    amount,
    commission,
    priority_expires_at,
    is_selected,
    created_by
)
VALUES
(
    (SELECT id FROM leads
      WHERE description = 'Защита товарного знака'),

    (SELECT id FROM agents
      WHERE email = 'ivanova@agent.test'),

    510000.00,
    10000.00,
    now() - INTERVAL '1 day',
    FALSE,

    (SELECT id FROM users
      WHERE email = 'manager2@lawfirm.test')
);


-- ============================================================
-- Настройки LLM
-- ============================================================

INSERT INTO llm_settings (
    provider,
    base_url,
    model,
    api_token_encrypted,
    is_active,
    updated_by
)
VALUES
(
    'yandexgpt',
    'https://example.invalid/llm',
    'test-model',
```

```
591     decode('746573742d746f6b656e', 'hex'),
592     TRUE,
593     (SELECT id FROM users
594       WHERE email = 'admin@lawfirm.test')
595 );
596 
597 
598 -- ============================================================
599 -- Лицензия
600 -- ============================================================
601 
602 INSERT INTO license_activations (
603     installation_id,
604     license_key_hash,
605     activation_secret,
606     token,
607     token_expires_at,
608     activated_by_user_id
609 )
610 VALUES
611 (
612     'a1b2c3d4-e5f6-7890-abcd-ef1234567890',
613     decode('0102030405060708', 'hex'),
614     decode('1112131415161718', 'hex'),
615     decode('2122232425262728', 'hex'),
616     now() + INTERVAL '24 hours',
617     (SELECT id FROM users
618       WHERE email = 'admin@lawfirm.test')
619 );
620 
621 COMMIT;
```

Листинг 5.1: seed_data.sql – тестовые данные

6. PL/pgSQL-функции и процедуры

Для реализации критически важных операций, соответствующих основным прецедентам системы, разработаны функции и процедуры на языке PL/pgSQL.

6.1. Регистрация лида

Процедура `register_lead` создаёт новый лид и при необходимости одновременно регистрирует первое предложение агента.

```
CREATE OR REPLACE PROCEDURE register_lead(
    p_client_id           INTEGER,
    p_opponent_id         INTEGER,
    p_description         TEXT,
    p_amount              NUMERIC(14,2),
    p_created_by          INTEGER,
    p_agent_id            INTEGER,
    p_offer_amount        NUMERIC(14,2),
    p_commission          NUMERIC(14,2),
    INOUT p_lead_id       INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM clients
        WHERE id = p_client_id
          AND merged_into_id IS NULL
    ) THEN
        RAISE EXCEPTION
            'Клиент % не найден или уже объединён с другим клие
               нтом',
            p_client_id;
    END IF;

    IF NOT EXISTS (
        SELECT 1
```

```
28          FROM opponents
29          WHERE id = p_opponent_id
30      ) THEN
31          RAISE EXCEPTION
32              'Оппонент % не найден',
33              p_opponent_id;
34      END IF;
35
36      INSERT INTO leads (
37          client_id,
38          opponent_id,
39          description,
40          amount,
41          status,
42          created_by,
43          updated_by
44      )
45      VALUES (
46          p_client_id,
47          p_opponent_id,
48          p_description,
49          p_amount,
50          'new',
51          p_created_by,
52          p_created_by
53      )
54      RETURNING id INTO p_lead_id;
55
56      IF p_agent_id IS NOT NULL THEN
57          IF p_offer_amount IS NULL AND p_amount IS NULL THEN
58              RAISE EXCEPTION
59                  'Для предложения агента должна быть указана сум
                        ма';
60          END IF;
61
62          INSERT INTO agent_offers (
63              lead_id,
64              agent_id,
65              amount,
66              commission,
67              created_by
```

```
        )
        VALUES (
            p_lead_id,
            p_agent_id,
            COALESCE(p_offer_amount, p_amount),
            COALESCE(p_commission, 0),
            p_created_by
        );
    END IF;
END;
$$;
```

6.2. Проверка конфликта интересов

Функция `check_conflict_of_interest` проверяет самого оппонента и связанные с ним организации на наличие среди текущих или недавних клиентов фирмы и сохраняет результат проверки.

```
CREATE OR REPLACE FUNCTION check_conflict_of_interest(
    p_lead_id     INTEGER,
    p_checked_by  INTEGER DEFAULT NULL
)
RETURNS TABLE (
    result VARCHAR(16),
    reason TEXT
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_opponent_id   INTEGER;
    v_archive_years INTEGER := 5;
    v_client_name   VARCHAR(512);
    v_depth         INTEGER;
    v_reason        TEXT;
BEGIN
    SELECT l.opponent_id
    INTO v_opponent_id
    FROM leads l
    WHERE l.id = p_lead_id;

    IF NOT FOUND THEN
```

```
        RAISE EXCEPTION 'Лид % не найден', p_lead_id;
    END IF;

    SELECT COALESCE(
        (br.definition ->> 'archive_years')::INTEGER,
        5
    )
    INTO v_archive_years
    FROM business_rules br
    WHERE br.rule_type = 'conflict_of_interest'
      AND br.is_active = TRUE
    ORDER BY br.id DESC
    LIMIT 1;

    IF NOT FOUND THEN
        v_archive_years := 5;
    END IF;

    WITH RECURSIVE related_opponents(id, depth) AS (
        SELECT v_opponent_id, 0

        UNION

        SELECT
            CASE
                WHEN a.parent_id = ro.id THEN a.child_id
                ELSE a.parent_id
            END,
            ro.depth + 1
        FROM related_opponents ro
        JOIN affiliations a
          ON a.parent_id = ro.id
          OR a.child_id = ro.id
        WHERE ro.depth < 5
    )
    SELECT
        COALESCE(c.full_name, c.short_name),
        ro.depth
    INTO
        v_client_name,
        v_depth
```

```
65      FROM related_opponents ro
66      JOIN opponents o
67        ON o.id = ro.id
68      JOIN clients c
69        ON c.inn IS NOT NULL
70       AND o.inn IS NOT NULL
71       AND c.inn = o.inn
72      WHERE c.merged_into_id IS NULL
73        AND EXISTS (
74            SELECT 1
75            FROM leads old_lead
76            WHERE old_lead.client_id = c.id
77              AND old_lead.id <> p_lead_id
78              AND (
79                  old_lead.status IN ('new', 'checking', 'active'
                        )
80                  OR old_lead.updated_at >=
81                      now() - make_interval(years =>
                            v_archive_years)
82              )
83        )
84      ORDER BY ro.depth
85      LIMIT 1;
86
87      IF FOUND THEN
88          IF v_depth = 0 THEN
89              v_reason := format(
90                  'Оппонент является текущим или недавним клиенто
                        м: %s',
91                  v_client_name
92              );
93          ELSE
94              v_reason := format(
95                  'Аффилированная организация является текущим ил
                        и недавним клиентом: %s',
96                  v_client_name
97              );
98          END IF;
99
100         INSERT INTO conflict_checks (
101             lead_id,
```

```
102             opponent_id,
103             result,
104             reason,
105             checked_by
106         )
107         VALUES (
108             p_lead_id,
109             v_opponent_id,
110             'blocked',
111             v_reason,
112             p_checked_by
113         );
114 
115         RETURN QUERY
116         SELECT
117             'blocked'::VARCHAR(16),
118             v_reason;
119 
120         RETURN;
121     END IF;
122 
123     v_reason := 'Конфликтов интересов не обнаружено';
124 
125     INSERT INTO conflict_checks (
126         lead_id,
127         opponent_id,
128         result,
129         reason,
130         checked_by
131     )
132     VALUES (
133         p_lead_id,
134         v_opponent_id,
135         'ok',
136         v_reason,
137         p_checked_by
138     );
139 
140     RETURN QUERY
141     SELECT
142         'ok'::VARCHAR(16),
```

```
        v_reason;
END;
$$;
```

6.3. Поиск дубликатов клиентов

Функция `find_client_duplicates` ищет возможные дубликаты клиента по ИНН, телефону, электронной почте и степени сходства имени.

```
CREATE OR REPLACE FUNCTION find_client_duplicates(
    p_client_id        INTEGER,
    p_min_similarity   NUMERIC DEFAULT 0.55
)
RETURNS TABLE (
    candidate_id     INTEGER,
    candidate_name   VARCHAR(512),
    match_type       VARCHAR(128),
    match_score      NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_phone VARCHAR(64);
    v_email VARCHAR(255);
    v_inn   VARCHAR(32);
    v_name  VARCHAR(512);
BEGIN
    SELECT
        c.phone,
        c.email,
        c.inn,
        COALESCE(c.full_name, c.short_name)
    INTO
        v_phone,
        v_email,
        v_inn,
        v_name
    FROM clients c
    WHERE c.id = p_client_id;

    IF NOT FOUND THEN
```

```
33          RAISE EXCEPTION 'Клиент % не найден', p_client_id;
34      END IF;
35
36      RETURN QUERY
37      SELECT
38          c.id,
39
40          COALESCE(
41              c.full_name,
42              c.short_name,
43              '(без имени)'
44          )::VARCHAR(512),
45
46          concat_ws(
47              ', ',
48              CASE
49                  WHEN v_inn IS NOT NULL
50                   AND c.inn = v_inn
51                  THEN 'ИНН'
52              END,
53              CASE
54                  WHEN v_phone IS NOT NULL
55                   AND c.phone = v_phone
56                  THEN 'телефон'
57              END,
58              CASE
59                  WHEN v_email IS NOT NULL
60                   AND lower(c.email) = lower(v_email)
61                  THEN 'e-mail'
62              END,
63              CASE
64                  WHEN v_name IS NOT NULL
65                   AND similarity(
66                          COALESCE(c.full_name, c.short_name, '')
                               ,
67                          v_name
68                      ) >= p_min_similarity
69                  THEN 'похожее имя'
70              END
71          )::VARCHAR(128),
72
```

```
        GREATEST(
            CASE
                WHEN v_inn IS NOT NULL AND c.inn = v_inn
                THEN 1.00
                ELSE 0
            END,

            CASE
                WHEN v_phone IS NOT NULL AND c.phone = v_phone
                THEN 0.95
                ELSE 0
            END,

            CASE
                WHEN v_email IS NOT NULL
                 AND lower(c.email) = lower(v_email)
                THEN 0.95
                ELSE 0
            END,

            CASE
                WHEN v_name IS NOT NULL
                THEN similarity(
                    COALESCE(c.full_name, c.short_name, ''),
                    v_name
                )
                ELSE 0
            END
        )::NUMERIC AS score

    FROM clients c

    WHERE c.id <> p_client_id
      AND c.merged_into_id IS NULL
      AND (
            (v_inn IS NOT NULL AND c.inn = v_inn)

         OR (v_phone IS NOT NULL AND c.phone = v_phone)

         OR (
             v_email IS NOT NULL
```

```
            AND lower(c.email) = lower(v_email)
        )

        OR (
            v_name IS NOT NULL
            AND similarity(
                COALESCE(c.full_name, c.short_name, ''),
                v_name
            ) >= p_min_similarity
        )
    )

    ORDER BY 4 DESC;
END;
$$;
```

Для нечёткого сравнения строк используется расширение `pg_trgm`:

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;
```

6.4. Объединение дубликатов клиентов

Процедура `merge_clients` переносит связанные лиды на основную запись клиента и помечает запись-дубликат как объединённую.

```
CREATE OR REPLACE PROCEDURE merge_clients(
    p_target_id  INTEGER,
    p_source_id  INTEGER,
    p_updated_by INTEGER DEFAULT NULL
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF p_target_id = p_source_id THEN
        RAISE EXCEPTION
            'Нельзя объединить клиента с самим собой';
    END IF;

    IF NOT EXISTS (
        SELECT 1
        FROM clients
        WHERE id = p_target_id
```

```
          AND merged_into_id IS NULL
    ) THEN
        RAISE EXCEPTION
            'Целевой клиент % не найден или уже объединён',
            p_target_id;
    END IF;

    IF NOT EXISTS (
        SELECT 1
        FROM clients
        WHERE id = p_source_id
          AND merged_into_id IS NULL
    ) THEN
        RAISE EXCEPTION
            'Исходный клиент % не найден или уже объединён',
            p_source_id;
    END IF;

    UPDATE leads
    SET
        client_id = p_target_id,
        updated_by = COALESCE(p_updated_by, updated_by)
    WHERE client_id = p_source_id;

    UPDATE clients
    SET
        merged_into_id = p_target_id,
        updated_by = COALESCE(p_updated_by, updated_by)
    WHERE merged_into_id = p_source_id;

    UPDATE clients
    SET
        merged_into_id = p_target_id,
        updated_by = COALESCE(p_updated_by, updated_by)
    WHERE id = p_source_id;
END;
$$;
```

6.5. Сравнение предложений агентов

Функция `compare_agent_offers` возвращает предложения агентов по выбранному лиду с расчётом чистой прибыли и признаком действующего приоритета.

```
CREATE OR REPLACE FUNCTION compare_agent_offers(
    p_lead_id INTEGER
)
RETURNS TABLE (
    offer_id              INTEGER,
    agent_id              INTEGER,
    agent_name            VARCHAR(255),
    amount                NUMERIC,
    commission            NUMERIC,
    net_profit            NUMERIC,
    priority_valid        BOOLEAN,
    priority_expires_at   TIMESTAMPTZ,
    is_selected           BOOLEAN
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM leads
        WHERE id = p_lead_id
    ) THEN
        RAISE EXCEPTION 'Лид % не найден', p_lead_id;
    END IF;

    RETURN QUERY
    SELECT
        o.id,
        a.id,
        a.full_name,
        o.amount,
        o.commission,

        o.amount - o.commission AS net_profit,

        COALESCE(
```

```
            o.priority_expires_at > now(),
            FALSE
        ) AS priority_valid,

        o.priority_expires_at,
        o.is_selected

    FROM agent_offers o

    JOIN agents a
      ON a.id = o.agent_id

    WHERE o.lead_id = p_lead_id

    ORDER BY
        COALESCE(
            o.priority_expires_at > now(),
            FALSE
        ) DESC,

        (o.amount - o.commission) DESC;
END;
$$;
```

6.6. Выбор предложения агента

Процедура `select_best_offer` фиксирует выбранное предложение агента и снимает признак выбора с остальных предложений этого лида.

```
CREATE OR REPLACE PROCEDURE select_best_offer(
    p_lead_id  INTEGER,
    p_offer_id INTEGER
)
LANGUAGE plpgsql
AS $$
BEGIN
    IF NOT EXISTS (
        SELECT 1
        FROM agent_offers
        WHERE id = p_offer_id
          AND lead_id = p_lead_id
```

```
    ) THEN
        RAISE EXCEPTION
            'Предложение % не относится к лиду %',
            p_offer_id,
            p_lead_id;
    END IF;

    UPDATE agent_offers
    SET is_selected = FALSE
    WHERE lead_id = p_lead_id;

    UPDATE agent_offers
    SET is_selected = TRUE
    WHERE id = p_offer_id;
END;
$$;
```

6.7. Снятие истёкших приоритетов

Процедура `revoke_expired_priorities` снимает приоритет с предложений агентов, для которых установленный срок уже истёк.

```
CREATE OR REPLACE PROCEDURE revoke_expired_priorities()
LANGUAGE plpgsql
AS $$
DECLARE
    v_count INTEGER;
BEGIN
    UPDATE agent_offers
    SET priority_expires_at = NULL
    WHERE priority_expires_at IS NOT NULL
      AND priority_expires_at <= now();

    GET DIAGNOSTICS v_count = ROW_COUNT;

    RAISE NOTICE
        'Снято просроченных приоритетов: %',
        v_count;
END;
$$;
```

7. Индексы и их обоснование

Все индексы были спроектированы на основе анализа прецедентов UC-1...UC-7 и функциональных требований FR-1...FR-12. В таблице 7 приведён перечень индексов с обоснованием их полезности.

Индекс	**Прецедент / FR**	**Обоснование**
`idx_clients_phone`	UC-3 (дедупликация)	Ускоряет поиск дубликатов клиентов по телефону – одному из ключевых идентификаторов.
`idx_clients_email`	UC-3 (дедупликация)	Аналогично, для поиска по e-mail.
`idx_clients_inn`	UC-2, UC-3	ИНН используется при проверке конфликта интересов (оппонент – юрлицо) и при поиске дубликатов юрлиц.
`idx_clients_full_name`	UC-3	Нечёткий поиск по ФИО через pg_trgm требует индекса.
`idx_clients_merged`	UC-3	Фильтрация объединённых клиентов при поиске дубликатов (частичный индекс).
`idx_leads_client_id`	UC-1, UC-4	Быстрый поиск всех лидов клиента при дедупликации и сравнении предложений.
`idx_leads_opponent_id`	UC-2	Проверка конфликта интересов выполняется по оппоненту.
`idx_leads_status`	UC-1, FR-8	Фильтрация лидов по статусу при автоматическом снятии приоритетов.

idx_leads_created	UC-1, отчёты	Сортировка лидов по дате создания в интерфейсе менеджера.
idx_offers_lead_id	UC-4	Сравнение всех предложений по лиду.
idx_offers_agent_id	FR-6	Поиск всех предложений конкретного агента.
idx_offers_expires	FR-8	Быстрый поиск просроченных приоритетов (частичный индекс).
idx_offers_selected	UC-4	Быстрый поиск выбранного предложения по лиду.
idx_opponents_inn	UC-2	Сопоставление оппонента с клиентом по ИНН.
idx_opponents_norm	UC-2	Поиск по нормализованному названию после LLM-обработки.
idx_affiliations_parent	FR-4	Поиск дочерних структур по материнской компании.
idx_affiliations_child	FR-4	Обратный поиск: материнская компания по дочерней.
idx_conflicts_lead	UC-2	История проверок по лиду.
idx_conflicts_opponent	UC-2	История проверок по оппоненту.
idx_rules_type	UC-5, FR-9	Поиск правил по типу при выполнении бизнес-логики.
idx_rules_def (GIN)	UC-5, FR-9	Полнотекстовый поиск внутри JSON-определений правил.
idx_license_expires	FR-12, NFR-9	Контроль срока действия лицензионных токенов.

7.1. Обоснование частичных индексов

Особое внимание уделено частичным (filtered) индексам, которые покрывают только подмножества строк и экономят место и время обслуживания:

- `idx_clients_phone WHERE phone IS NOT NULL` – большинство клиентов имеют телефон, но частичный индекс позволяет не хранить NULL-значения в B-дереве.
- `idx_offers_expires WHERE priority_expires_at IS NOT NULL` – для операции FR-8 (снятие просроченных приоритетов) нужны только активные приоритеты; после истечения срока запись исключается из индекса.
- `idx_rules_active WHERE is_active = TRUE` – при выполнении правил требуются только активные правила.

Заключение

В рамках второго этапа курсовой работы спроектирована и реализована реляционная база данных PostgreSQL для информационной системы юридической фирмы.

Основные результаты:

1. Построена ER-модель, включающая 12 сущностей и несколько отношений вида «многие-ко-многим» (агенты ↔ лиды через предложения, клиенты ↔ оппоненты через проверки конфликтов).

2. Разработана даталогическая модель, преобразованная в DDL-скрипт создания базы данных с использованием типов PostgreSQL (JSONB для правил, BYTEA для хэшей и шифрованных токенов, UUID для идентификаторов установок).

3. Обеспечена целостность данных на уровне DDL (первичные и внешние ключи, уникальные ограничения, CHECK-ограничения) и на уровне триггеров (автоматическое обновление временных меток, расчёт срока приоритета агента по бизнес-правилу, автоматическая проверка конфликта интересов при создании лида, защита от удаления клиентов с активными лидами).

4. Реализованы скрипты `create_database.sql`, `drop_database.sql` и `seed_data.sql`, покрывающие все основные сценарии предметной области (дубликаты клиентов, конфликты интересов, аффилированные оппоненты, несколько предложений агентов).

5. Разработаны PL/pgSQL-функции и процедуры для критически важных операций: `check_conflict_of_interest` (UC-2), `find_client_duplicates` (UC-3), `compare_agent_offers` и `select_best_of` (UC-4), `revoke_expired_priorities` (FR-8), `merge_clients` (UC-3).

6. Спроектированы индексы с обоснованием для каждого прецедента; применены частичные индексы для оптимизации часто используемых выборок.

Реализованная база данных готова к интеграции со слоем Spring Boot на следующем этапе курсовой работы. Все функции и процедуры могут быть вызваны из Java-кода через Spring Data JPA (`@NamedStoredProcedureQuery`) или JDBC-шаблоны.